

# VAE

March 8, 2026

## 1 PS2.1: Variational Autoencoders (VAE)

In this notebook, you will implement a variational autoencoder and a conditional variational autoencoder with slightly different architectures and apply them to the popular MNIST handwritten dataset. An autoencoder seeks to learn a latent representation of our training images by using unlabeled data and learning to reconstruct its inputs. The **variational autoencoder** extends this model by adding a probabilistic spin to the encoder and decoder, allowing us to sample from the learned distribution of the latent space to generate new images at inference time.

**Credits** This notebook is based on a problem set by Justin Johnson (<https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/assignment6.html>).

### 1.1 Setup and Imports

```
[1]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
import random
import math

# Set device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

# Set random seeds for reproducibility
torch.manual_seed(42)
np.random.seed(42)
```

Using device: cuda

## 1.2 Useful Functions for Debugging

```
[2]: def reset_seed(number):  
    """  
    Reset random seed to the specific number  
  
    Inputs:  
    - number: A seed number to use  
    """  
    random.seed(number)  
    torch.manual_seed(number)  
    return  
  
def rel_error(x, y, eps=1e-10):  
    """  
    Compute the relative error between a pair of tensors x and y,  
    which is defined as:  
  
    
$$\text{rel\_error}(x, y) = \frac{\max_i |x_i - y_i|}{\max_i |x_i| + \max_i |y_i| + \text{eps}}$$
  
  
    Inputs:  
    - x, y: Tensors of the same shape  
    - eps: Small positive constant for numeric stability  
  
    Returns:  
    - rel_error: Scalar giving the relative error between x and y  
    """  
    """ returns relative error between x and y """  
    top = (x - y).abs().max().item()  
    bot = (x.abs() + y.abs()).clamp(min=eps).max().item()  
    return top / bot  
  
def show_images(images):  
    images = torch.reshape(  
        images, [images.shape[0], -1]  
    ) # images reshape to (batch_size, D)  
    sqrtn = int(math.ceil(math.sqrt(images.shape[0])))  
    sqrtimg = int(math.ceil(math.sqrt(images.shape[1])))  
  
    fig = plt.figure(figsize=(sqrtn, sqrtn))  
    gs = gridspec.GridSpec(sqrtn, sqrtn)  
    gs.update(wspace=0.05, hspace=0.05)  
  
    for i, img in enumerate(images):
```

```

    ax = plt.subplot(gs[i])
    plt.axis("off")
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.set_aspect("equal")
    plt.imshow(img.reshape([sqrting, sqrting, 1]), 'Greys_r')
return

```

### 1.3 Load MNIST Dataset

In this assignment, we will be working on the MNIST dataset, which has 60,000 training and 10,000 test images. Each picture contains a centered image of white digit on black background (0 through 9). This was one of the first datasets used to train convolutional neural networks and it is fairly easy – a standard CNN model can easily exceed 99% accuracy.

To simplify our code here, we will use the PyTorch MNIST wrapper, which downloads and loads the MNIST dataset. See the [documentation](#) for more information about the interface.

```

[3]: # Load Data
# We use a simple transform to convert data to tensor
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(root='./data', train=True, transform=transform,
    ↪download=True)
test_dataset = datasets.MNIST(root='./data', train=False, transform=transform,
    ↪download=True)

# Configuration
BATCH_SIZE = 128

train_loader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE,
    ↪shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=BATCH_SIZE,
    ↪shuffle=False)

```

```

Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

```

```

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz

```

```

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz to ./data/MNIST/raw/train-images-idx3-ubyte.gz

```

```

100%|          | 9.91M/9.91M [00:00<00:00, 22.1MB/s]

```

```

Extracting ./data/MNIST/raw/train-images-idx3-ubyte.gz to ./data/MNIST/raw

```

```

Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz

```

Failed to download (trying next):  
HTTP Error 404: Not Found

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz to ./data/MNIST/raw/train-labels-idx1-ubyte.gz

100%| | 28.9k/28.9k [00:00<00:00, 1.33MB/s]

Extracting ./data/MNIST/raw/train-labels-idx1-ubyte.gz to ./data/MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz

Failed to download (trying next):  
HTTP Error 404: Not Found

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz to ./data/MNIST/raw/t10k-images-idx3-ubyte.gz

100%| | 1.65M/1.65M [00:00<00:00, 10.9MB/s]

Extracting ./data/MNIST/raw/t10k-images-idx3-ubyte.gz to ./data/MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz

Failed to download (trying next):  
HTTP Error 404: Not Found

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz to ./data/MNIST/raw/t10k-labels-idx1-ubyte.gz

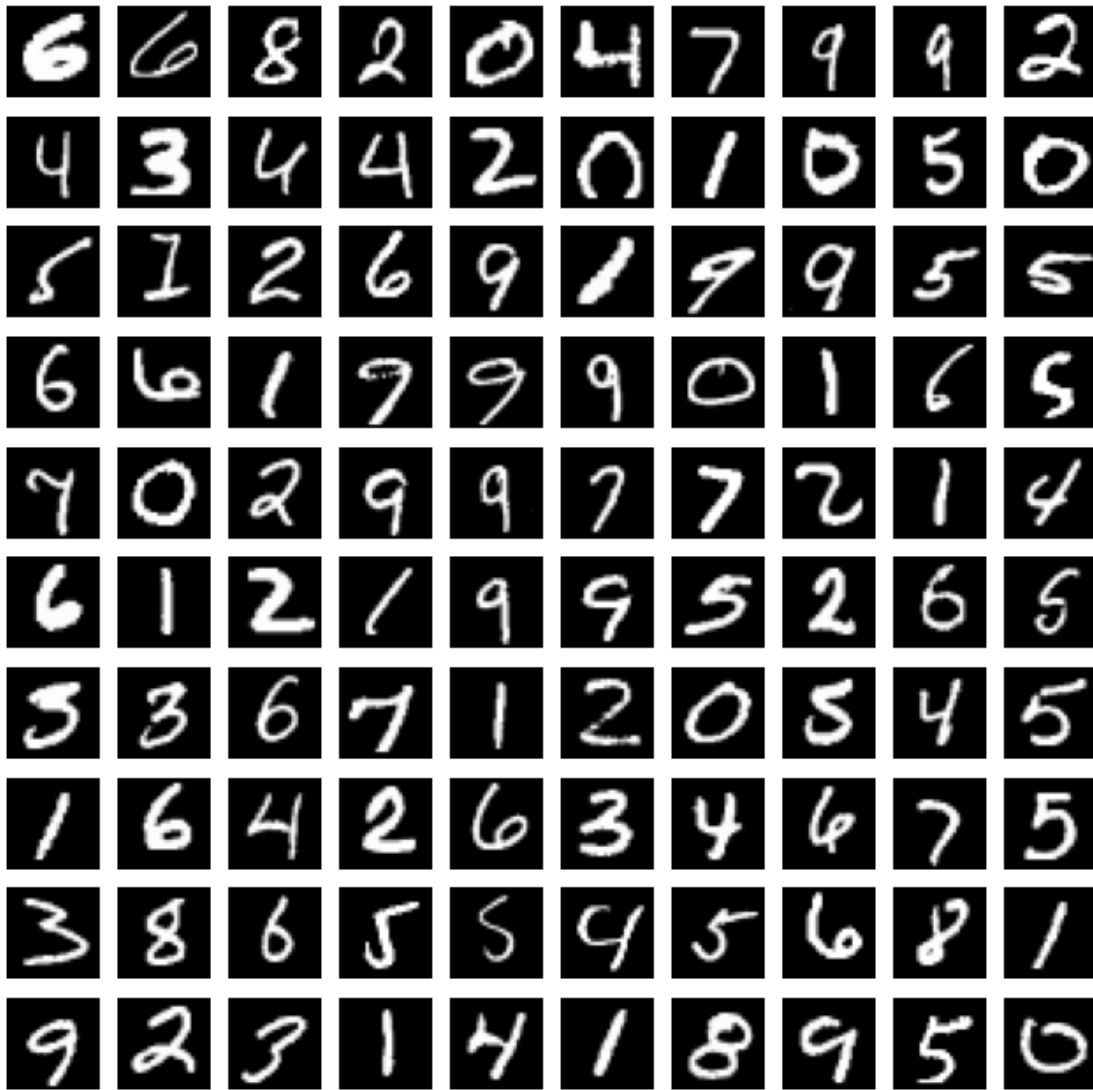
100%| | 4.54k/4.54k [00:00<00:00, 3.46MB/s]

Extracting ./data/MNIST/raw/t10k-labels-idx1-ubyte.gz to ./data/MNIST/raw

## 1.4 Visualize Dataset

```
[4]: figure = plt.figure(figsize=(8, 8))
     cols, rows = 10, 10
     for i in range(1, cols * rows + 1):
         sample_idx = torch.randint(len(train_dataset), size=(1,)).item()
         img, label = train_dataset[sample_idx]
         figure.add_subplot(rows, cols, i)
```

```
plt.axis("off")
plt.imshow(img.squeeze(), cmap="gray")
plt.show()
```



## 1.5 Fully Connected VAE

Our first VAE implementation will consist solely of fully connected layers. We'll take the  $1 \times 28 \times 28$  shape of our input and flatten the features to create an input dimension size of 784. In this section you'll define the Encoder and Decoder models in the VAE class, and implement the reparametrization trick, forward pass, and loss function to train your first VAE.

## 1.6 FC-VAE Encoder

Now let's start building our fully-connected VAE network. We'll start with the encoder, which will take our images as input (after flattening C,H,W to D shape) and pass them through a three Linear+ReLU layers. We'll use this hidden dimension representation to predict both the posterior  $\mu$  and posterior log-variance using two separate linear layers (both shape (N,Z)).

Note that we are calling this the 'logvar' layer because we'll use the log-variance (instead of variance or standard deviation) to stabilize training. This will specifically matter more when you compute reparametrization and the loss function later.

*Define an `encoder`, `hidden_dim(H)`, `mu_layer`, and `logvar_layer` in the initialization of the VAE class. Use `nn.Sequential` to define the encoder, and separate Linear layers for the `mu` and `logvar` layers. In all of these layers, `H` will be a hidden dimension you set and will be the same across all encoder and decoder layers. Architecture for the encoder is described below:*

- Flatten (Hint: `nn.Flatten`)
- Fully connected layer with input size 784 (`input_size`) and output size H
- ReLU
- Fully connected layer with input\_size H and output size H
- ReLU
- Fully connected layer with input\_size H and output size H
- ReLU

## 1.7 FC-VAE Decoder

We'll now define the decoder, which will take the latent space representation and generate a reconstructed image. The architecture is as follows:

- Fully connected layer with input size as the latent size (Z) and output size H
- ReLU
- Fully connected layer with input\_size H and output size H
- ReLU
- Fully connected layer with input\_size H and output size H
- ReLU
- Fully connected layer with input\_size H and output size 784 (`input_size`)
- Sigmoid
- Unflatten (`nn.Unflatten`)

## 1.8 Reparametrization

Now we'll apply a reparametrization trick in order to estimate the posterior  $z$  during our forward pass, given the  $\mu$  and  $\sigma^2$  estimated by the encoder. A simple way to do this could be to simply generate a normal distribution centered at our  $\mu$  and having a std corresponding to our  $\sigma^2$ . However, we would have to backpropagate through this random sampling that is not differentiable. Instead, we sample initial random data  $\epsilon$  from a fixed distribution, and compute  $z$  as a function of  $(\epsilon, \sigma^2, \mu)$ . Specifically:

$$z = \mu + \sigma\epsilon$$

We can easily find the partial derivatives w.r.t  $\mu$  and  $\sigma^2$  and backpropagate through  $z$ . If  $\epsilon = \mathcal{N}(0, 1)$ , then it's easy to verify that the result of our forward pass calculation will be a distribution

centered at  $\mu$  with variance  $\sigma^2$ .

Implement reparametrization in VAE class and verify your mean and std error are at or less than  $1e-4$ .

## 1.9 FC-VAE Forward

Complete the VAE class by writing the forward pass. The forward pass should pass the input image through the encoder to calculate the estimation of mu and logvar, reparametrize to estimate the latent space z, and finally pass z into the decoder to generate an image.

```
[5]: class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=16):
        super(VAE, self).__init__()
        self.input_dim = input_dim # H*W
        self.latent_dim = latent_dim # Z
        self.hidden_dim = hidden_dim # H_d
        self.encoder = None
        self.mu_layer = None
        self.logvar_layer = None
        self.decoder = None

    #####
    # TODO: Implement the fully-connected encoder architecture described in
    # the notebook. Specifically, self.encoder should be a network that
    # inputs a batch of input images of shape (N, 1, H, W) into a batch of
    # hidden features of shape (N, H_d). Set up self.mu_layer and
    # self.logvar_layer to be a pair of linear layers that map the hidden
    # features into estimates of the mean and log-variance of the posterior
    # over the latent vectors; the mean and log-variance estimates will
    # both be tensors of shape (N, Z).
    #####

    self.encoder = nn.Sequential(
        nn.Flatten(),
        nn.Linear(input_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim),
```

```

        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU()
    )
    self.mu_layer = nn.Linear(hidden_dim, latent_dim)
    self.logvar_layer = nn.Linear(hidden_dim, latent_dim)

    #####
    #                                     END OF YOUR CODE
    #
    #####

    #####
    # TODO: Implement the fully-connected decoder architecture described in
    # the notebook. Specifically, self.decoder should be a network that
    # takes as input a batch of latent vectors of shape (N, Z) and outputs a tensor of
    # estimated images of shape (N, 1, H, W).
    #
    #####

    self.decoder = nn.Sequential(
        nn.Linear(latent_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, input_dim),
        nn.Sigmoid(),
        nn.Unflatten(1, (1, 28, 28))
    )

    #####
    #                                     END OF YOUR CODE
    #
    #####

```



```
def reparameterize(self, mu, logvar):
    """
    Implements the reparameterization trick:
     $z = \mu + \sigma * \epsilon$ 
    """

    # TODO: Implement the reparameterization trick

    # 1. Calculate standard deviation (sigma) from logvar

    # 2. Generate random noise (epsilon) with the same shape as sigma

    # 3. Return  $\mu + \sigma * \epsilon$ 

    # Hint: use torch.randn_like() for epsilon or torch.randn()

    sigma = torch.exp(0.5 * logvar)
    epsilon = torch.randn_like(sigma)
    z = mu + sigma * epsilon
    return z

    # END OF YOUR CODE

def forward(self, x):
    x_hat = None
    mu = None
    logvar = None

    # TODO: Implement the forward pass by following these steps

    # 1. Pass the input batch through the encoder model to get posterior
    #      mu and logvariance
```

```

    # 2. Reparametrize to compute the latent vector z
    ↪ #
    # 3. Pass z through the decoder to reconstruct x
    ↪ #
    ↪ #####

    img_embedding = self.encoder(x)
    mu = self.mu_layer(img_embedding)
    logvar = self.logvar_layer(img_embedding)
    z = self.reparameterize(mu, logvar)
    x_hat = self.decoder(z)

    ↪ #####
    ↪ #                                END OF YOUR CODE
    ↪ #
    ↪ #####

    return x_hat, mu, logvar

```

### 1.10 Initialize Model

```

[6]: # Configuration
LEARNING_RATE = 1e-3
LATENT_DIM = 16

# Initialize Model
vae_model = VAE(latent_dim=LATENT_DIM).to(device)
optimizer = optim.Adam(vae_model.parameters(), lr=LEARNING_RATE)

```

### 1.11 Sanity Check

Verify your mean and std error are at or less than 1e-4.

```

[7]: reset_seed(42)
latent_size = 16
size = (1, latent_size)
mu = torch.zeros(size)
logvar = torch.ones(size)

z = vae_model.reparameterize(mu, logvar)

expected_mean = torch.FloatTensor([-0.2254])
expected_std = torch.FloatTensor([2.0351])
z_mean = torch.mean(z, dim=-1)

```

```

z_std = torch.std(z, dim=-1)
assert z.size() == size

print('Mean Error', rel_error(z_mean, expected_mean))
print('Std Error', rel_error(z_std, expected_std))

```

Mean Error 6.313092501083968e-05  
Std Error 6.677690713968148e-06

## 1.12 Loss Function

Before we're able to train our final model, we'll need to define our loss function. As seen below, the loss function for VAEs contains two terms: A reconstruction loss term (left) and KL divergence term (right).

$$-E_{Z \sim q_\phi(z|x)}[\log p_\theta(x|z)] + D_{KL}(q_\phi(z|x), p(z))$$

Note that this is the negative of the variational lowerbound shown in lecture—this ensures that when we are minimizing this loss term, we're maximizing the variational lowerbound. The reconstruction loss term can be computed by simply using the binary cross entropy loss between the original input pixels and the output pixels of our decoder (Hint: `nn.functional.binary_cross_entropy`). The KL divergence term works to force the latent space distribution to be close to a prior distribution (we're using a standard normal gaussian as our prior).

To help you out, we've derived an unvectorized form of the KL divergence term for you. Suppose that  $q_\phi(z|x)$  is a  $Z$ -dimensional diagonal Gaussian with mean  $\mu_{z|x}$  of shape  $(Z,)$  and standard deviation  $\sigma_{z|x}$  of shape  $(Z,)$ , and that  $p(z)$  is a  $Z$ -dimensional Gaussian with zero mean and unit variance. Then we can write the KL divergence term as:

$$D_{KL}(q_\phi(z|x), p(z)) = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_{z|x}^2)_j - (\mu_{z|x})_j^2 - (\sigma_{z|x})_j^2)$$

It's up to you to implement a vectorized version of this loss that also operates on minibatches. You should average the loss across samples in the minibatch.

Implement `loss_function` and verify your implementation below.

```

[17]: # Reconstruction + KL Divergence losses summed over all elements and batch
def loss_function(x_hat, x, mu, logvar):
    loss = None

    #####
    # TODO: Compute negative variational lowerbound loss as described above.
    #
    #####

    reconstruction_loss = F.binary_cross_entropy(x_hat, x, reduction='sum')
    kl_divergence = -0.5 * torch.sum(1 + logvar - mu.pow(2) - torch.exp(logvar))
    loss = reconstruction_loss + kl_divergence
    loss /= x.size(0)

```

```

└─
↪#####
#                                END OF YOUR CODE                                └─
↪ #
└─
↪#####
return loss

```

### 1.13 Sanity Check

Your relative error should be less than or equal to  $1e-4$ .

```

[18]: size = (1,16)

image = torch.sigmoid(torch.FloatTensor([[2,5], [6,7]]).unsqueeze(0).
↪unsqueeze(0))
image_hat = torch.sigmoid(torch.FloatTensor([[1,10], [9,3]]).unsqueeze(0).
↪unsqueeze(0))

expected_out = torch.tensor(9.0096)
mu, logvar = torch.ones(size), torch.zeros(size)
out = loss_function(image, image_hat, mu, logvar)
print('Loss error', rel_error(expected_out,out))

```

Loss error 9.63336064549868e-05

### 1.14 Train model

Let's now train our VAE!

Training for 10 epochs should take ~2 minutes and your loss should be less than 120.

```

[19]: def train(epoch):
    vae_model.train()
    train_loss = 0
    for batch_idx, (data, _) in enumerate(train_loader):
        data = data.to(device)
        optimizer.zero_grad()

        # Forward pass
        recon_batch, mu, logvar = vae_model(data)

        # Compute loss
        loss = loss_function(recon_batch, data, mu, logvar)

        # Backward pass
        loss.backward()
        train_loss += loss.item()

```

```

optimizer.step()

print(f'====> Epoch: {epoch} Loss: {loss:.4f}')

# Training
EPOCHS = 10
for epoch in range(1, EPOCHS + 1):
    train(epoch)

====> Epoch: 1 Loss: 110.8711
====> Epoch: 2 Loss: 107.0891
====> Epoch: 3 Loss: 107.3553
====> Epoch: 4 Loss: 113.9277
====> Epoch: 5 Loss: 105.9840
====> Epoch: 6 Loss: 103.1935
====> Epoch: 7 Loss: 110.8896
====> Epoch: 8 Loss: 110.3314
====> Epoch: 9 Loss: 108.0392
====> Epoch: 10 Loss: 113.4371

```

## 1.15 Visualization

### 1.15.1 1. Reconstruction

Let's look at how well the model reconstructs inputs from the test set.

```

[20]: def visualize_reconstruction():
    vae_model.eval()
    data, _ = next(iter(test_loader))
    data = data.to(device)
    recon, _, _ = vae_model(data)

    # Plot original vs reconstructed
    fig, axes = plt.subplots(2, 8, figsize=(10, 3))
    for i in range(8):
        # Original
        axes[0, i].imshow(data[i].cpu().view(28, 28), cmap='gray')
        axes[0, i].axis('off')
        # Reconstructed
        axes[1, i].imshow(recon[i].cpu().detach().view(28, 28), cmap='gray')
        axes[1, i].axis('off')
    plt.suptitle("Top: Original | Bottom: Reconstructed")
    plt.show()

visualize_reconstruction()

```

Top: Original | Bottom: Reconstructed



### 1.15.2 2. Generation

Let's look at how well the model can randomly generate new samples.

```
[21]: z = torch.randn(10, latent_size).to(device)
import matplotlib.gridspec as gridspec
vae_model.eval()
samples = vae_model.decoder(z).data.cpu().numpy()

fig = plt.figure(figsize=(10, 1))
gspec = gridspec.GridSpec(1, 10)
gspec.update(wspace=0.05, hspace=0.05)
for i, sample in enumerate(samples):
    ax = plt.subplot(gspec[i])
    plt.axis('off')
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.set_aspect('equal')
    plt.imshow(sample.reshape(28,28), cmap='Greys_r')
```



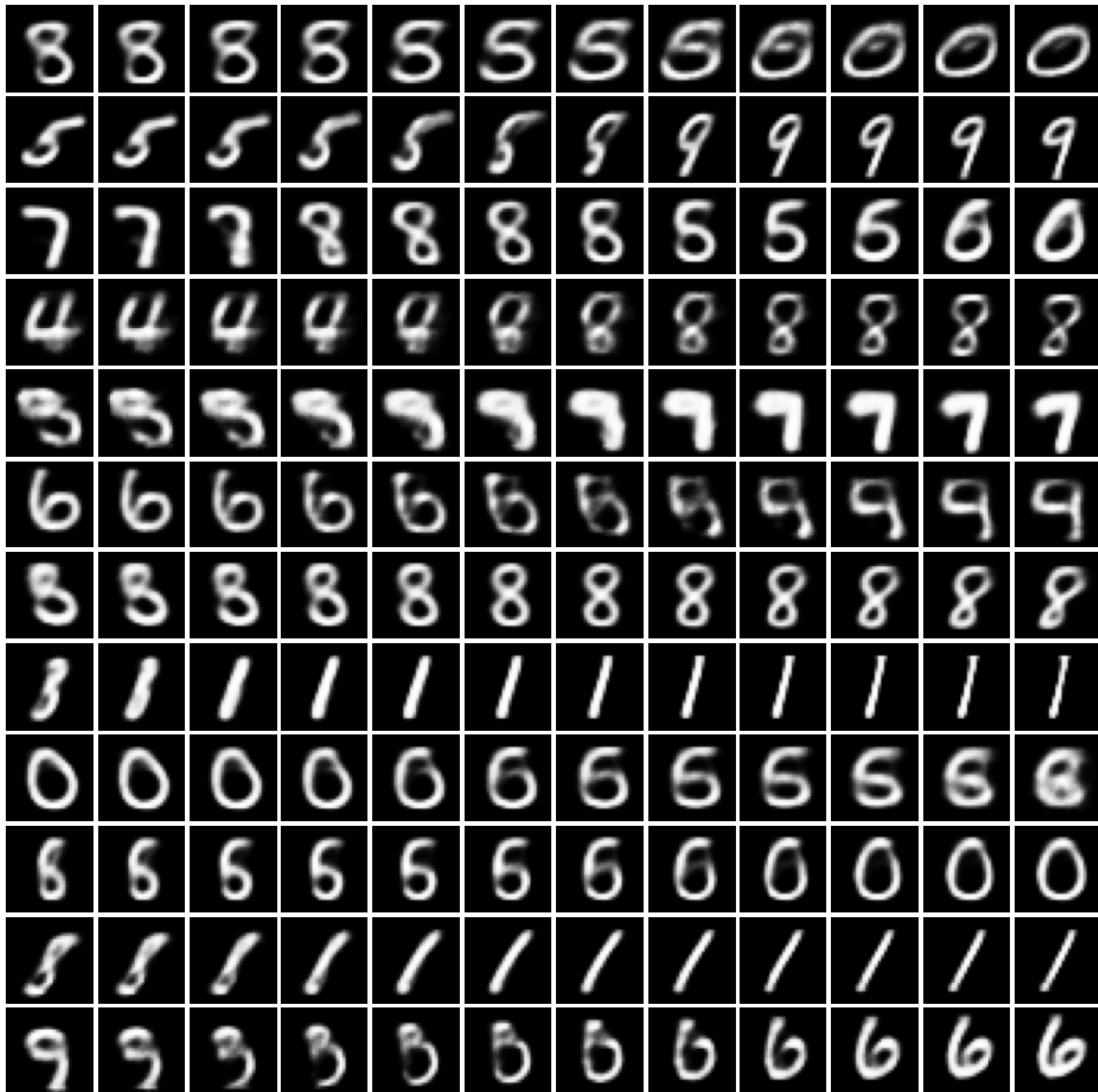
### 1.15.3 3. Latent Space Interpolation

As a final visual test of our trained VAE model, we can perform interpolation in latent space. We generate random latent vectors  $z_0$  and  $z_1$ , and linearly interpolate between them; we run each interpolated vector through the trained generator to produce an image.

Each row of the figure below interpolates between two random vectors. For the most part the model should exhibit smooth transitions along each row, demonstrating that the model has learned

something nontrivial about the underlying spatial structure of the digits it is modeling.

```
[22]: S = 12
z0 = torch.randn(S,LATENT_DIM , device=device)
z1 = torch.randn(S, LATENT_DIM, device=device)
w = torch.linspace(0, 1, S, device=device).view(S, 1, 1)
z = (w * z0 + (1 - w) * z1).transpose(0, 1).reshape(S * S, LATENT_DIM)
x = vae_model.decoder(z)
show_images(x.data.cpu())
```



## 2 Conditional FC-VAE

The second model you'll develop will be very similar to the FC-VAE, but with a slight conditional twist to it. We'll use what we know about the labels of each MNIST image, and *condition* our latent space and image generation on the specific class. Instead of  $q_\phi(z|x)$  and  $p_\phi(x|z)$  we have  $q_\phi(z|x, c)$  and  $p_\phi(x|z, c)$

This will allow us to do some powerful conditional generation at inference time. We can specifically choose to generate more 1s, 2s, 9s, etc. instead of simply generating new digits randomly.

### 2.1 Define Network with class input

Our CVAE architecture will be the same as our FC-VAE architecture, except we'll now add a one-hot label vector to both the x input (in our case, the flattened image dimensions) and the z latent space.

If our one-hot vector is called `c`, then `c[label] = 1` and `c = 0` elsewhere.

For the CVAE class, use the same FC-VAE architecture implemented in the last network with the following modifications:

1. Modify the first linear layer of your `encoder` to take in not only the flattened input image, but also the one-hot label vector `c`
2. Modify the first layer of your `decoder` to project the latent space + one-hot vector to the `hidden_dim`
3. Lastly, implement the `forward` pass to combine the flattened input image with the one-hot vectors (`torch.cat`) before passing them to the `encoder` and combine the latent space with the one-hot vectors (`torch.cat`) before passing them to the `decoder`

```
[33]: class CVAE(nn.Module):
    def __init__(self, input_dim=784, num_classes=10, hidden_dim=400,
        latent_dim=16):
        super(CVAE, self).__init__()
        self.input_dim = input_dim # H*W
        self.num_classes = num_classes
        self.latent_dim = latent_dim # Z
        self.hidden_dim = hidden_dim # H_d
        self.encoder = None
        self.mu_layer = None
        self.logvar_layer = None
        self.decoder = None

    # TODO: Define a FC encoder as described in the notebook that
    # transforms #
    # the image--after flattening and now adding our one-hot class vector
    # (N, #
    # H*W + C)--into a hidden_dimension (N, H_d) feature space, and a final
    #
```



```

        # two layers that project that feature space to posterior mu and
        posterior#
        # log-variance estimates of the latent space (N, Z)
        #
        #####

        self.encoder = nn.Sequential(
            nn.Linear(input_dim + num_classes, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        self.mu_layer = nn.Linear(hidden_dim, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim, latent_dim)

        #####
        #                                END OF YOUR CODE
        #
        #####

        #####
        # TODO: Define a fully-connected decoder as described in the notebook
        that#
        # transforms the latent space (N, Z + C) to the estimated images of
        shape #
        # (N, 1, H, W).
        #
        #####

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim + num_classes, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim),
            nn.Sigmoid(),
            nn.Unflatten(1, (1, 28, 28))

```

```

    )

    #####
    #                                     END OF YOUR CODE
    #
    #####

    def reparameterize(self, mu, logvar):
        """
        Implements the reparameterization trick:
         $z = \mu + \sigma * \epsilon$ 
        """

        #####
        # TODO: Implement the reparameterization trick
        #
        # 1. Calculate standard deviation (sigma) from logvar
        #
        # 2. Generate random noise (epsilon) with the same shape as sigma
        #
        # 3. Return  $\mu + \sigma * \epsilon$ 
        #
        # Hint: use torch.randn_like() for epsilon or torch.randn()
        #
        #####

        sigma = torch.exp(0.5 * logvar)
        epsilon = torch.randn_like(sigma)
        z = mu + sigma * epsilon
        return z

    #####
    #                                     END OF YOUR CODE
    #
    #####

    def forward(self, x, c):
        x_hat = None
        mu = None
        logvar = None

```

```

#####
# TODO: Implement the forward pass by following these steps
#
# 1. Pass the input batch through the encoder model to get posterior
#
#      mu and logvariance
#
# 2. Reparametrize to compute the latent vector z
#
# 3. Pass z through the decoder to reconstruct x
#
#####
x_flattened = x.view(x.shape[0], -1)

img_embedding = self.encoder(torch.cat([x_flattened, c], dim=1))
mu = self.mu_layer(img_embedding)
logvar = self.logvar_layer(img_embedding)
z = self.reparameterize(mu, logvar)
x_hat = self.decoder(torch.cat([z, c], dim=1))

#####
#
#
#
#####
#
#
#
#####
return x_hat, mu, logvar

```

## 2.2 Initialize Model

```

[34]: # Initialize Model
cvae_model = CVAE(latent_dim=LATENT_DIM).to(device)
optimizer = optim.Adam(cvae_model.parameters(), lr=LEARNING_RATE)

```

## 2.3 Train model

Using the same training script, let's now train our CVAE!

Training for 10 epochs should take ~2 minutes and your loss should be less than 120.

```

[35]: def train(epoch):
    cvae_model.train()
    train_loss = 0
    for batch_idx, (data, labels) in enumerate(train_loader):
        data = data.to(device)

```

```

labels = labels.to(device)
one_hot_labels = F.one_hot(labels, num_classes=10)
optimizer.zero_grad()

# Forward pass
recon_batch, mu, logvar = cvae_model(data, one_hot_labels)

# Compute loss
loss = loss_function(recon_batch, data, mu, logvar)

# Backward pass
loss.backward()
train_loss += loss.item()
optimizer.step()

print(f'====> Epoch: {epoch} Loss: {loss:.4f}')

# Training
EPOCHS = 10
for epoch in range(1, EPOCHS + 1):
    train(epoch)

```

```

====> Epoch: 1 Loss: 144.9704
====> Epoch: 2 Loss: 125.8050
====> Epoch: 3 Loss: 122.4211
====> Epoch: 4 Loss: 118.8091
====> Epoch: 5 Loss: 109.8826
====> Epoch: 6 Loss: 110.3495
====> Epoch: 7 Loss: 102.9392
====> Epoch: 8 Loss: 112.4735
====> Epoch: 9 Loss: 110.9800
====> Epoch: 10 Loss: 105.6737

```

## 2.4 Visualize Results

Now that we've trained our CVAE, let's conditionally generate some new data! This time, we can specify the class we want to generate by adding our one hot matrix of class labels. We use `torch.eye` to create an identity matrix, which effectively gives us one label for each digit. When you run the cell below, you should get one example per digit. Each digit should be reasonably distinguishable (it is ok to run this cell a few times to save your best results).

```

[36]: z = torch.randn(10, latent_size)
      c = torch.eye(10, 10) # [one hot labels for 0-9]
      import matplotlib.gridspec as gridspec
      z = torch.cat((z,c), dim=-1).to(device)
      cvae_model.eval()
      samples = cvae_model.decoder(z).data.cpu().numpy()

```

```

fig = plt.figure(figsize=(10, 1))
gspec = gridspec.GridSpec(1, 10)
gspec.update(wspace=0.05, hspace=0.05)
for i, sample in enumerate(samples):
    ax = plt.subplot(gspec[i])
    plt.axis('off')
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.set_aspect('equal')
    plt.imshow(sample.reshape(28, 28), cmap='Greys_r')

```



### 2.4.1 Latent Space Interpolation

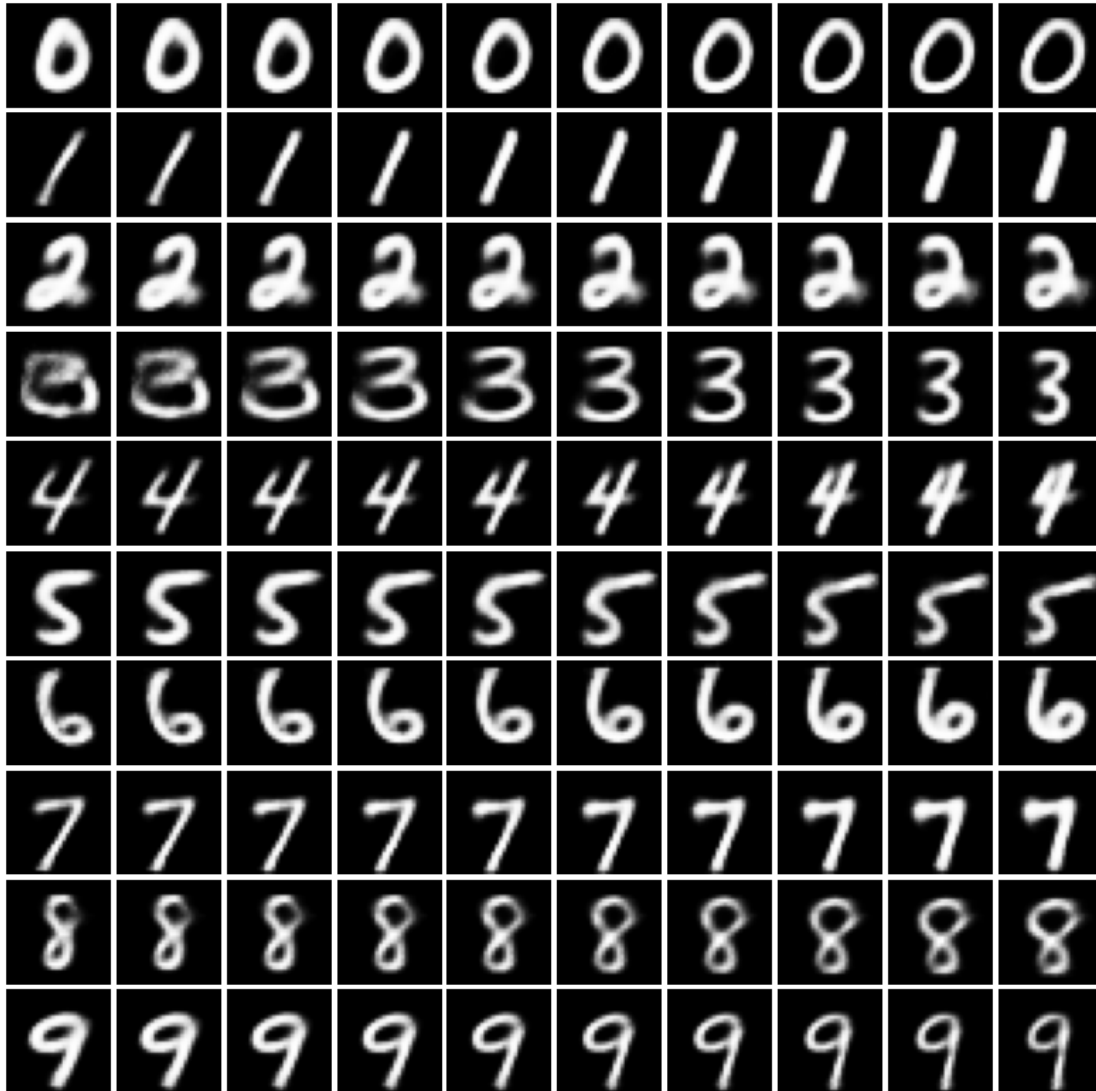
As a final visual test of our trained CVAE model, we can perform interpolation in latent space. We generate random latent vectors  $z_0$  and  $z_1$ , and linearly interpolate between them; we run each interpolated vector through the trained generator to produce an image.

The  $i$ 'th row of the figure below interpolates between two random vectors and is conditioned on the sample label  $i$ . For the most part the model should exhibit smooth transitions along each row, demonstrating that the model has learned something nontrivial about the underlying spatial structure of the digits it is modeling.

```

[37]: z0 = torch.randn(10, LATENT_DIM)
      z1 = torch.randn(10, LATENT_DIM)
      w = torch.linspace(0, 1, 10).view(10, 1, 1)
      z = (w * z0 + (1 - w) * z1).transpose(0, 1).reshape(10 * 10, LATENT_DIM)
      c = torch.eye(10, 10).unsqueeze(1).repeat(1, 10, 1).reshape(-1, 10)
      z = torch.cat((z, c), dim=-1).to(device)
      cvae_model.eval()
      samples = cvae_model.decoder(z).data.cpu()
      show_images(samples)

```



## 2.5 Convert Notebook to PDF

```
[ ]: # Generate and download a PDF for this notebook.
# Please provide the full path of the notebook file below (we have provided a
↳ default filename, but this might not match yours!)
#
# *If you run into problems running this code, please see this guide*: https://docs.google.com/document/d/1Y5SpnkX4Cp7bPL72CI9B5YSv5tUCiDHxVg3w0L5MXXU/edit?usp=sharing
notebook_path = './VAE.ipynb' # UPDATE THIS

import os
file_name = notebook_path.split('/')[-1]
```

```

get_ipython().system("apt update && apt install texlive-xetex_
↳texlive-fonts-recommended texlive-generic-recommended")
get_ipython().system("pip install py pandoc")
get_ipython().system("apt-get install texlive texlive-xetex texlive-latex-extra_
↳pandoc")
get_ipython().system("jupyter nbconvert --to PDF '{}'.format(notebook_path))

from google.colab import files
files.download(notebook_path.split('.')[0]+'pdf')

```

Reading package lists... Done

**E:** Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)

**E:** Unable to lock directory /var/lib/apt/lists/

**W:** Problem unlinking the file /var/cache/apt/pkgcache.bin - RemoveCaches (13: Permission denied)

**W:** Problem unlinking the file /var/cache/apt/srcpkgcache.bin - RemoveCaches (13: Permission denied)

Collecting py pandoc

Downloading py pandoc-1.16.2-py3-none-any.whl.metadata (16 kB)

Downloading py pandoc-1.16.2-py3-none-any.whl (19 kB)

Installing collected packages: py pandoc

Successfully installed py pandoc-1.16.2

**E:** Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)

**E:** Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?

[NbConvertApp] Converting notebook ./VAE.ipynb to PDF

[NbConvertApp] Support files will be in VAE\_files/

[NbConvertApp] Making directory ./VAE\_files

[NbConvertApp] Writing 120946 bytes to notebook.tex

[NbConvertApp] Building PDF

[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']